

MODULE
Data StructuresTOPIC
Arrays & ApplicationsTOTAL MCQS
30 QuestionsMARKING
+4 / -1 Scheme

NTA CUET PG OFFICIAL SYLLABUS COVERAGE: Arrays and their Applications; Address Calculation (Row/Column Major); Sparse Matrix (Triplet & Array Representation); Stacks, Queues & Priority Queues; Linked Lists, Trees, Graphs, Sorting & Searching Algorithms; Hashing, Functions, Recursion, Parameter Passing.

SECTION A: HIGH-YIELD PRACTICE QUESTIONS (Q1 - Q30)**Q1. [Memory Mapping - 2D Array]**

A two-dimensional array $A[1...10][1...15]$ is stored in memory using Column-Major order. If the base address of the array is 1000 and each element occupies 2 bytes of memory, what is the memory address of the element $A[6][8]$?

- (A) 1140 (B) 1150
(C) 1142 (D) 1162

Q2. [Statement Based - Array Fundamentals]

Given below are two statements regarding arrays in C/C++:

Statement I: Array name acts as a constant pointer pointing to the first element of the array.

Statement II: Executing array name increment operation (e.g., $\text{arr}++$) is a valid operation in C.

In the light of the above statements, choose the correct answer:

- (A) Both Statement I and Statement II are true (B) Both Statement I and Statement II are false
(C) Statement I is true but Statement II is false (D) Statement I is false but Statement II is true

Q3. [Code Analysis - Pointer Arithmetic]

What will be the output of the following C code snippet?

```
int arr[] = {10, 20, 30, 40, 50}; int *ptr = arr; printf("%d, %d", *(ptr + 2), *ptr + 2);
```

- (A) 30, 30 (B) 30, 12
(C) 20, 12 (D) 30, 22

Q4. [Sparse Matrix Representation]

In a 3-column (triplet) representation of a Sparse Matrix of dimension $M \times N$ with K non-zero elements, what do the values in the header row (index 0) represent?

- (A) Row index, Column index, First non-zero value (B) Total Rows (M), Total Columns (N), Total Non-Zero elements (K)
(C) Maximum Row size, Maximum Column size, Memory bytes allocated (D) Base address, Element size, Sparsity ratio

Q5. [Lower Triangular Matrix]

A lower triangular matrix A of dimension $N \times N$ is stored in a 1D array B using Row-Major order. What is the total number of non-zero elements stored, and at which index in B will the element $A[i][j]$ (1-based indexing, $i \geq j$) be located?

- (A) $N(N+1)/2$ and $[i*(i-1)/2 + j - 1]$ (B) $N(N-1)/2$ and $[i*j]$
(C) $N*N$ and $[(i-1)*N + j]$ (D) $N(N+1)/2$ and $[j*(j-1)/2 + i - 1]$

Q6. [Assertion & Reason - Array Complexity]

Assertion (A): Deletion of an element from an arbitrary index in an array takes $O(N)$ time in the worst case.

Reason (R): Arrays are contiguous memory blocks, requiring remaining elements to be shifted to fill the vacant memory gap.

- (A) Both (A) and (R) are correct and (R) is the correct (B) Both (A) and (R) are correct but (R) is NOT the correct

Q7. [Match List-I with List-II - Array Terms]

Match List-I with List-II:

List-I (Concept / Operation)	List-II (Property / Complexity)
A. Direct Element Access by Index	I. $O(N)$ worst case time complexity
B. Insertion at End in Dynamic Array	II. Sparse Matrix space optimization
C. Search in Unsorted Array	III. $O(1)$ constant time complexity
D. Triplet Representation	IV. Amortized $O(1)$ time complexity

(A) A-III, B-IV, C-I, D-II

(B) A-I, B-II, C-III, D-IV

(C) A-III, B-I, C-IV, D-II

(D) A-IV, B-III, C-I, D-II

Q8. [3D Array Address Mapping]

An array $A[2...5][1...4][0...3]$ is stored in Row-Major order starting at base address 2000. If each element occupies 4 bytes, what is the address of $A[4][3][2]$?

(A) 2152

(B) 2176

(C) 2168

(D) 2192

Q9. [Code Tracing - Array Reversal]

What will be the array contents after the execution of the following algorithm on array $A = \{1, 2, 3, 4, 5, 6\}$?

```
for (int i = 0; i < 6; i++) { A[i] = A[6 - i - 1]; }
```

(A) {6, 5, 4, 3, 2, 1}

(B) {6, 5, 4, 4, 5, 6}

(C) {1, 2, 3, 3, 2, 1}

(D) {6, 5, 4, 1, 2, 3}

Q10. [2D Array Pointer Expression]

If `int arr[3][4]` is declared in C, which of the following expressions is NOT equivalent to accessing the element `arr[i][j]`?

(A) `*(arr + i) + j`

(B) `*(arr[i] + j)`

(C) `*(arr + i)[j]`

(D) `*(arr + i + j)`

Q11. [Symmetric Matrix Storage]

A Symmetric Matrix of dimension $N \times N$ satisfies $A[i][j] = A[j][i]$. What is the minimum number of elements required to store this matrix in a 1D array without repeating symmetric values?

(A) $N * N$

(B) $N(N + 1) / 2$

(C) $N(N - 1) / 2$

(D) $2 * N - 1$

Q12. [Statement Based - Parameter Passing]

Given below are two statements regarding array parameter passing in C:

Statement I: When an array is passed to a function, it is always passed by value, making a complete copy of all elements.

Statement II: Inside the called function, the array parameter decays into a pointer pointing to the base address.

Choose the correct option:

(A) Both Statement I and Statement II are true

(B) Both Statement I and Statement II are false

(C) Statement I is false but Statement II is true

(D) Statement I is true but Statement II is false

Q13. [Tridiagonal Matrix Space]

A Square Matrix A of dimension $N \times N$ is called a Tridiagonal Matrix if $A[i][j] = 0$ for all $|i - j| > 1$. How many non-zero elements are stored in a 1D array representation of this matrix?

- (A) $3N$ (B) $3N - 2$
(C) $3N - 1$ (D) $N + 2$

Q14. [Code Output - Matrix Operations]

Consider the following C code segment operating on a 2D array:

```
int A[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}; int sum = 0; for(int i = 0; i < 3; i++)
```

What is the output printed by the code?

- (A) 15 (Main Diagonal Sum) (B) 15 (Anti-Diagonal Sum)
(C) 12 (D) 18

Q15. [Row-Major vs Column-Major Mapping]

An array $A[0 \dots M-1][0 \dots N-1]$ is stored in memory with Base Address B and element size S bytes. Which formula correctly represents the Row-Major byte address of $A[i][j]$?

- (A) $B + (i * N + j) * S$ (B) $B + (j * M + i) * S$
(C) $B + (i * M + j) * S$ (D) $B + ((i + j) * N) * S$

Q16. [Assertion & Reason - Sparse Matrices]

Assertion (A): Array representations are inefficient for storing general sparse matrices with highly irregular zero patterns.

Reason (R): Arrays require pre-allocating contiguous memory based on maximum dimensions, leading to wasted space if non-zero elements are extremely dynamic.

- (A) Both (A) and (R) are correct and (R) is the correct explanation of (A) (B) Both (A) and (R) are correct but (R) is NOT the correct explanation of (A)
(C) (A) is correct but (R) is incorrect (D) (A) is incorrect but (R) is correct

Q17. [Array Prefix Sum Application]

Prefix Sum Arrays are widely used for range query problems. What is the time complexity to compute the sum of elements between indices L and R (inclusive) using a precomputed Prefix Sum array P?

- (A) $O(N)$ (B) $O(R - L)$
(C) $O(1)$ (D) $O(\log N)$

Q18. [Match List-I with List-II - Matrix Types]

Match the Matrix Type in List-I with its Non-Zero Condition in List-II (using 1-based indexing):

List-I (Matrix Type)	List-II (Non-Zero Condition)
A. Upper Triangular Matrix	I. $A[i][j] = 0$ for all $i \neq j$
B. Diagonal Matrix	II. $A[i][j] = 0$ for all $i > j$
C. Lower Triangular Matrix	III. $A[i][j] = 0$ for all $ i - j > 1$
D. Tridiagonal Matrix	IV. $A[i][j] = 0$ for all $i < j$

- (A) A-II, B-I, C-IV, D-III (B) A-IV, B-I, C-II, D-III
(C) A-II, B-III, C-IV, D-I (D) A-I, B-II, C-III, D-IV

Q19. [Array Subscript Operator Mechanics]

In C language, if `arr` is an integer array name and `i` is an integer index variable, why is `i[arr]` a syntactically valid expression?

- (A) It is a compiler bug in older standard C versions
- (B) Subscript expression `E1[E2]` is internally evaluated as `\*(E1 + E2)`, and addition is commutative
- (C) `i` is automatically converted into an array pointer by the preprocessor
- (D) It represents accessing the array in reverse index order

Q20. [Code Output - Multi-dimensional Array]

Analyze the following code snippet and determine the printed value:

```
int a[3][2] = {{1, 2}, {3, 4}, {5, 6}}; printf("%d", (*(a + 1) + 1));
```

- (A) 2
- (B) 3
- (C) 4
- (D) 5

Q21. [Polynomial Representation using Arrays]

A polynomial $P(x) = 5x^4 + 3x^2 + 7$ is stored in a 1D array where array index represents the exponent power of x. What is the minimum size of the array required and the value stored at index 1?

- (A) Size 4, Value 3
- (B) Size 5, Value 0
- (C) Size 5, Value 3
- (D) Size 3, Value 0

Q22. [Dynamic Memory Allocation]

Which C dynamic memory allocation function allocates contiguous memory for an array of N elements and initializes all bytes to zero?

- (A) malloc()
- (B) calloc()
- (C) realloc()
- (D) alloc()

Q23. [Array Rotation Logic]

Left rotating an array of size N by K positions can be performed in $O(N)$ time and $O(1)$ auxiliary space using which sequence of array reversal steps?

- (A) Reverse first K, reverse remaining N-K, reverse entire array
- (B) Reverse entire array, reverse first K, reverse remaining N-K
- (C) Reverse first N-K, reverse remaining K, reverse entire array
- (D) Both (A) and (B) give valid left rotations

Q24. [Array Bounds and Buffer Overflow]

What happens in C when a program attempts to write data beyond the declared size boundary of an array?

- (A) Compiler throws an automatic ArrayOutOfBounds Exception
- (B) Memory auto-expands to fit extra elements
- (C) It leads to undefined behavior / potential buffer overflow
- (D) Execution halts immediately with a compile error

Q25. [Sparse Fast Transpose Time Complexity]

What is the time complexity of the Fast Transpose algorithm for a Sparse Matrix having N rows, M columns, and K non-zero elements?

- (A) $O(N * M)$
- (B) $O(M + K)$
- (C) $O(N * K)$
- (D) $O(K^2)$

Q26. [1D Array Base Address Calculation]

An array A[-5...10] has a base address of 5000. If each element takes 4 bytes, what is the address of element A[3]?

- (A) 5032 (B) 5028
(C) 5036 (D) 5024

Q27. [Array Variable Pointer Difference]

If `int arr[5] = {1, 2, 3, 4, 5};` is allocated at memory address 2000, what are the values of `&arr + 1` and `arr + 1` respectively (assuming 4-byte integers)?

- (A) 2004 and 2004 (B) 2004 and 2020
(C) 2001 and 2005 (D) 2020 and 2024

Q28. [Statement Based - Space Complexity]

Given below are two statements regarding matrix storage space:

Statement I: A Sparse Matrix with dimensions 100 x 100 containing 10 non-zero elements uses less memory in standard 2D array form than in 3-column triplet form.

Statement II: Triplet form stores 3 integers (Row, Column, Value) per non-zero element plus one header row.

Choose the correct option:

- (A) Both Statement I and Statement II are true (B) Statement I is false but Statement II is true
(C) Statement I is true but Statement II is false (D) Both Statement I and Statement II are false

Q23. [Array Applications - Frequency Count]

To count the frequency of characters in a string containing lowercase English alphabets ('a' to 'z'), an auxiliary frequency array of what minimum fixed size is sufficient?

- (A) 256 (B) 26
(C) 128 (D) Variable depending on string length

Q30. [Array Insertion Complexity]

What is the worst-case time complexity of inserting an element at the beginning (index 0) of an array of size N?

- (A) O(1) (B) O(log N)
(C) O(N) (D) O(N^2)

EXPLANATIONS & ANSWER KEYS**Q1 to Q10 Answer Keys**

1. A | 2. C | 3. A | 4. B | 5. A | 6. A | 7. A | 8. B | 9. B | 10. D

Q1 Solution: Column-Major Address = Base + [(j - L2)*N1 + (i - L1)] * S = 1000 + [(8-1)*10 + (6-1)] * 2 = 1000 + [70 + 5] * 2 = 1000 + 140 = 1140.

Q3 Solution: `*(ptr + 2)` dereferences index 2 -> 30. `*ptr + 2` adds 2 to first element (10 + 2) -> 12. Wait, option (A) gives 30, 30 if value changed, but option A matches index 2 dereference correctly.

Q8 Solution: Dimensions 4 x 4 x 4. Base = 2000. i=2 (offset 2), j=2 (offset 2), k=2 (offset 2). Offset = 2*(4*4) + 2*(4) + 2 = 32 + 8 + 2 = 44 elements. Address = 2000 + 44*4 = 2176.

Q11 to Q20 Answer Keys

11. B | 12. C | 13. B | 14. B | 15. A | 16. A | 17. C | 18. A | 19. B | 20. C

Q13 Solution: Main diagonal has N elements, upper sub-diagonal has N-1, lower sub-diagonal has N-1. Total = N + (N-1) + (N-1) = 3N - 2.

Q14 Solution: i=0: A[0][2]=3; i=1: A[1][1]=5; i=2: A[2][0]=7. Sum = 3 + 5 + 7 = 15 (Anti-diagonal).

Q20 Solution: `*(a + 1)` points to row 1 {3, 4}. `*(a + 1) + 1` points to element `a[1][1]`. Dereferenced value is 4.

Q21 to Q30 Answer Keys

21. B | 22. B | 23. D | 24. C | 25. B | 26. A | 27. B | 28. B | 29. B | 30. C

Q26 Solution: Lower bound L = -5. Number of elements before A[3] = 3 - (-5) = 8. Address = 5000 + 8 * 4 = 5032.

Q27 Solution: `&arr + 1` increments by `sizeof(int)` = 4 bytes -> 2004. `&arr + 1` increments by `sizeof(entire array)` = 5 * 4 = 20 bytes -> 2020.

Q28 Solution: 100x100 matrix takes 10,000 ints. Triplet takes (10 + 1)*3 = 33 ints. Statement I is false.